

Resoluções das Questões Complementares — Capítulo 2

Obra: C: Como Programar, 6ª Edição

Autores: Paul Deitel & Harvey Deitel

Abordagem: Progressão Incremental e Rigor de Engenharia de Software

Apresentação Pedagógica

Olá! É um prazer trazer o complemento oficial com os exercícios que haviam ficado de fora da nossa primeira rodada de resoluções do Capítulo 2. Mantendo o nosso compromisso firme com a **abordagem pedagógica incremental**, todos os códigos a seguir foram elaborados utilizando exclusivamente a matéria dada até o fim do Capítulo 2: declaração sequencial de tipos inteiros, I/O formatado (printf e scanf), operadores aritméticos elementares e avaliação condicional simples por meio da instrução if de seleção única. Não utilizamos o bloco else, estruturas de laço (while ou for) ou funções customizadas, já que estes recursos pertencem estritamente aos capítulos vindouros.

1. Resoluções Passo a Passo e Códigos-Fonte

Exercício 2.21 — Desenho de Formas com Asteriscos

Enunciado: Escreva um programa que imprima uma caixa, uma oval, uma seta e um diamante utilizando asteriscos (*).

Análise Científica: Este exercício desenvolve a percepção espacial do programador e a capacidade de mapear uma matriz bidimensional de texto no terminal serializado utilizando a sequência de escape de nova linha (\n).

```
#include <stdio.h>
```

```
int main(void) {
    printf("***** *** * *\n");
    printf("* * * * * * * *\n");
    printf("* * * * ***** * *\n");
    printf("* * * * * * * *\n");
    printf("* * * * * * * *\n");
    printf("* * * * * * * *\n");
    printf("* * * * * * * *\n");
    printf("***** *** * *\n");

    return 0;
}
```

Boa Prática de Programação 2.1: Manter strings longas perfeitamente alinhadas visualmente dentro do próprio código-fonte ajuda imensamente na manutenção de interfaces de texto e na detecção precoce de erros de posicionamento.

Exercício 2.22 — Análise Estatística de Saída de Código

Enunciado: O que o seguinte código imprime?

```
printf("\n**\n***\n****\n*****\n");
```

Explicação Didática: O fluxo de execução lê a string de controle de formato de caractere em caractere. Cada asterisco literal é renderizado de imediato na tela. Sempre que o token de controle de escape `\n` é encontrado, o cursor do console é instruído a descer uma linha e se reposicionar na coluna inicial à esquerda. O processo gera um triângulo de altura e base de 5 unidades.

Saída Gerada no Terminal:

```
*
**
***
****
*****
```

Exercício 2.24 — Verificação de Paridade (Ímpar ou Par)

Enunciado: Escreva um programa que leia um número inteiro e então determine e imprima se ele é par ou ímpar.

Abordagem Incremental: Como o comando `else` ainda não foi ensinado, resolvemos o fluxo lógico utilizando duas estruturas de controle `if` completamente independentes, baseadas em testes booleanos mutuamente exclusivos sobre o operador de resto (%).

```
#include <stdio.h>
```

```
int main(void) {
    int numero;

    printf("Digite um número inteiro: ");
    scanf("%d", &numero);

    /* Condição para número Par */
    if (numero % 2 == 0) {
        printf("O número %d é PAR.\n", numero);
    }

    /* Condição para número Ímpar */
    if (numero % 2 != 0) {
        printf("O número %d é ÍMPAR.\n", numero);
    }

    return 0;
}
```

```
}
```

Exercício 2.25 — Renderização de Iniciais em Bloco

Enunciado: Imprima suas iniciais em letras maiúsculas no sentido vertical, de cima para baixo na página. Construa cada letra de sua inicial com a própria letra que ela representa.

Resolução Didática: Mapeamos as iniciais da nossa marca pedagógica (P de Paul, D de Deitel) de forma estilizada usando strings repetidas.

```
#include <stdio.h>

int main(void) {
    /* Letra P */
    printf("PPPPPPPP\n");
    printf("P        P\n");
    printf("P        P\n");
    printf("PPPPPPPP\n");
    printf("P\n");
    printf("P\n");
    printf("P\n\n");

    /* Letra D */
    printf("DDDDDDDD\n");
    printf("D        D\n");
    printf("D        D\n");
    printf("D        D\n");
    printf("D        D\n");
    printf("DDDDDDDD\n");

    return 0;
}
```

Exercício 2.26 — Relação de Multiplicidade entre Inteiros

Enunciado: Escreva um programa que leia dois inteiros e determine e imprima se o primeiro é múltiplo do segundo.

Análise Científica: Um inteiro a é considerado múltiplo de um inteiro b se a divisão de a por b gerar um quociente exato, isto é, se o resto obtido for estritamente igual a zero ($a \% b == 0$).

```
#include <stdio.h>

int main(void) {
    int num1, num2;

    printf("Digite dois números inteiros: ");
    scanf("%d%d", &num1, &num2);

    if (num1 % num2 == 0) {
```

```

        printf("%d é múltiplo de %d.\n", num1, num2);
    }

    if (num1 % num2 != 0) {
        printf("%d não é múltiplo de %d.\n", num1, num2);
    }

    return 0;
}

```

Exercício 2.27 — Padrão Reticular de Tabuleiro de Xadrez

Enunciado: Desenhe um padrão tipo tabuleiro de xadrez utilizando instruções printf e depois exiba o mesmo padrão com o menor número de instruções possível.

Abordagem A (Instruções Explícitas por Linha — 8 printf):

```
#include <stdio.h>
```

```

int main(void) {
    printf("* * * * * \n");
    printf(" * * * * * \n");
    printf("* * * * * \n");
    printf(" * * * * * \n");
    printf("* * * * * \n");
    printf(" * * * * * \n");
    printf("* * * * * \n");
    printf(" * * * * * \n");

    return 0;
}

```

Abordagem B (Instrução Compactada Otimizada — 1 printf):

```
#include <stdio.h>
```

```

int main(void) {
    printf("* * * * * \n * * * * * \n * * * * * \n * * * * * \n * * * * * \n * * * * * \n * * * * * \n * * * * * \n");

    return 0;
}

```

Exercício 2.28 — Engenharia de Software: Erros Fatais vs. Não Fatais

Enunciado: Diga a diferença entre os termos erro fatal e erro não-fatal. Por que você poderia desejar a ocorrência de um erro fatal em vez de um erro não-fatal?

Categoria de Erro	Comportamento do Sistema	Impacto na Depuração
Erro Fatal	Interrompe e encerra abruptamente a execução do aplicativo em tempo de execução de imediato.	Altamente perceptível. Expõe o ponto exato da quebra do código, facilitando a rápida ação de correção do engenheiro.
Erro Não Fatal	O programa executa até a conclusão natural, porém gera e salva dados logicamente incorretos.	Sutil e traiçoeiro. Pode passar despercebido em testes preliminares, corrompendo fluxos de dados de forma silenciosa.

Justificativa Técnica: Um desenvolvedor prefere amplamente a ocorrência de um **erro fatal** em ambiente de desenvolvimento. Por causar a queda imediata do sistema, ele atua como um sinalizador explícito do problema. Erros não fatais mascaram bugs de lógica complexos e podem chegar de forma catastrófica até o cliente final.

Exercício 2.29 — Mapeamento Numérico de Caracteres (Tabela ASCII)

Enunciado: Escreva um programa que imprima os equivalentes inteiros de algumas letras maiúsculas, minúsculas, dígitos e símbolos especiais.

Explicação Científica: Computadores manipulam internamente caracteres puramente como valores numéricos binários discretos codificados. Ao utilizarmos o especificador de tipo e conversão %d passando um caractere literal entre aspas simples (ex: 'A'), forçamos a função printf a ler a representação inteira correspondente definida pelo mapa de caracteres do sistema (padrão ASCII/ISO-8859).

```
#include <stdio.h>
```

```
int main(void) {
    printf("Equivalente numérico de 'A': %d\n", 'A');
    printf("Equivalente numérico de 'B': %d\n", 'B');
    printf("Equivalente numérico de 'C': %d\n", 'C');
    printf("Equivalente numérico de 'a': %d\n", 'a');
    printf("Equivalente numérico de 'b': %d\n", 'b');
    printf("Equivalente numérico de 'c': %d\n", 'c');
    printf("Equivalente numérico de '0': %d\n", '0');
    printf("Equivalente numérico de '1': %d\n", '1');
    printf("Equivalente numérico de '2': %d\n", '2');
    printf("Equivalente numérico de '$': %d\n", '$');
    printf("Equivalente numérico de '*': %d\n", '*');
    printf("Equivalente numérico de '+': %d\n", '+');
    printf("Equivalente numérico de '/': %d\n", '/');
    printf("Equivalente numérico de ' ': %d\n", ' '); /* Caractere de
    espaço */
}
```

```
    return 0;
}
```

2. Código-Fonte LaTeX Completo para Compilação Acadêmica

Abaixo está o documento em notação técnica estrita pronto para ser colado em ambientes de compilação:

```
\documentclass[12pt]{article}
\usepackage[utf8]{inputenc}
\usepackage[brazil]{babel}
\usepackage{amsmath}
\usepackage{booktabs}
\usepackage{listings}
\usepackage{geometry}
\geometry{a4paper, margin=1in}

\lstset{
    language=C,
    basicstyle=\ttfamily\small,
    breaklines=true,
    keywordstyle=\bfseries,
    commentstyle=\itshape,
    showstringspaces=false
}

\title{Exercícios Complementares Resolvidos -- Capítulo 2\\ \large
C: Como Programar (6ª edição, Ed. Deitel)}
\author{Prof. Paul Deitel}
\date{\today}

\begin{document}
\maketitle

\section*{Apresentação ao Pedagógica}
Este documento visa complementar as resoluções do Capítulo 2,
cobrindo estritamente os exercícios omitidos inicialmente, seguindo
o padrão incremental de aprendizado sem uso de estruturas de
controle complexas.

\subsection*{Exercício 2.21 -- Formas com Asteriscos}
\begin{lstlisting}
#include
int main(void) {
```

```

printf("***** *** * *\n");
printf("* * * * * *\n");
printf("* * * * * *\n");
printf("* * * * * *\n");
printf("* * * * * *\n");
printf("* * * * * *\n");
printf("* * * * * *\n");
printf("* * * * * *\n");
printf("***** *** * *\n");
return 0;
}
\end{lstlisting}

```

\subsection*{Exerc\'icio 2.22 -- An\'alise de Sa\'ida}

O c\'odigo imprime sequencialmente asteriscos que formam um tri\'angulo reto de base e altura 5, quebrando a linha a cada s\'imbolo \textbackslash n.

```

\subsection*{Exerc\'icio 2.24 -- Par ou \'Impar}
\begin{lstlisting}
#include
int main(void) {
    int numero;
    printf("Digite um numero inteiro: ");
    scanf("%d", &numero);
    if (numero % 2 == 0) {
        printf("O numero %d e PAR.\n", numero);
    }
    if (numero % 2 != 0) {
        printf("O numero %d e IMPAR.\n", numero);
    }
    return 0;
}
\end{lstlisting}

```

```

\subsection*{Exerc\'icio 2.25 -- Iniciais em Bloco}
\begin{lstlisting}
#include
int main(void) {
    printf("PPPPPPPP\nP          P\nPPPPPPPPPP\nP\nP\n\n");
    printf("DDDDDDDD\nD          D\nD          D\nDDDDDDDD\n");
    return 0;
}
\end{lstlisting}

```

```

\subsection*{Exerc\'icio 2.26 -- M\'ultiplos}
\begin{lstlisting}
#include

```

```

int main(void) {
    int num1, num2;
    printf("Digite dois inteiros: ");
    scanf("%d%d", &num1, &num2);
    if (num1 % num2 == 0) {
        printf("%d e multiplo de %d.\n", num1, num2);
    }
    if (num1 % num2 != 0) {
        printf("%d nao e multiplo de %d.\n", num1, num2);
    }
    return 0;
}
\end{lstlisting}

```

\subsection*{Exerc\'icio 2.27 -- Padr\~ao de Xadrez}

```

\begin{lstlisting}
#include
int main(void) {
    printf(" * * * * * \n * * * * * \n * * * * * \n * * *
 * * * * * \n");
    return 0;
}
\end{lstlisting}

```

\subsection*{Exerc\'icio 2.28 -- Erros Fatais vs. N\~ao Fatais}

Erros fatais causam a interrup\c{c}\~ao abrupta e imediata da execu\c{c}\~ao do programa, facilitando o isolamento e corre\c{c}\~ao do bug por parte do programador. Erros n\~ao fatais s\~ao l\'ogicos e silenciosos; o programa roda at\'e o fim gerando dados incorretos, o que os torna muito dif\'iceis de detectar.

\subsection*{Exerc\'icio 2.29 -- Valor Inteiro de Caracteres}

```

\begin{lstlisting}
#include
int main(void) {
    printf("Valor de 'A': %d\n", 'A');
    printf("Valor de 'a': %d\n", 'a');
    printf("Valor de '0': %d\n", '0');
    printf("Valor de ' ': %d\n", ' ');
    return 0;
}
\end{lstlisting}

```

\end{document}